Universidad del Valle de Guatemala

Construcción de Compiladores · Sección 10

De MiniLang a WebAssembly

Anatomía visual de un compilador

Actividad 01 · Generación de contenido

DragonsSlayers 2.0

Pablo Daniel Barillas Moreno

Hugo Daniel Barillas Ajín

Ernesto Ascencio

Guion técnico y de exposición · Duración objetivo: 14:40

Octubre de 2026

1. Propósito y enfoque

El video sigue un único programa mientras cambia de representación: caracteres, tokens, árbol sintáctico, símbolos, WebAssembly textual y ejecución. La frase conductora es: *un compilador no comprende el programa de una sola vez; lo transforma hasta hacer explícito su significado*.

Audiencia	Estudiantes de computación que saben programar, pero aún perciben el compilador como una caja negra.
Objetivo	Diferenciar las fases principales y observar cómo una expresión se convierte en instrucciones WebAssembly ejecutables.
Duración	14 minutos y 40 segundos, más o menos 20 segundos según ritmo natural.
Formato	1920x1080, 30 fps, voz principal a -14 LUFS aproximados y música discreta.
Licencia	Creative Commons Attribution; solo recursos propios, CC BY o dominio público.

2. Guion cronológico

1. El resultado antes del proceso 00:00–00:42 | Voz: Pablo

Visual: Aparición breve de Pablo. Corte al clip Manim `HookScene`. En pantalla aparece el programa y luego el resultado 42.
Audio: Ambiente electrónico muy tenue. Efecto de señal al recorrer la flecha y acorde corto al aparecer el resultado.
Narración: Escribimos programas todos los días y esperamos que la computadora los ejecute. Pero para una máquina, esta línea todavía no es una multiplicación ni una declaración: es solamente una secuencia de caracteres. En este video construiremos un lenguaje pequeño llamado MiniLang y seguiremos este programa desde el texto original hasta un módulo real de WebAssembly. Al final obtendremos este resultado: cuarenta y dos. La pregunta es qué tuvo que ocurrir para llegar hasta aquí.

2. Abrir la caja negra 00:42–01:32 | Voz: Pablo

Visual: Clip `PipelineScene`. La señal recorre Código, Lexer, Parser, Semántica, WAT y Runtime.
Narración: Un compilador no transforma todo el programa de una sola vez. Lo procesa mediante una cadena de representaciones. El lexer convierte caracteres en tokens. El parser organiza esos tokens en un árbol. El análisis semántico comprueba que los nombres y operaciones tengan sentido. Después, un generador produce instrucciones de otro lenguaje. En nuestro caso, ese destino será WebAssembly. Cada etapa resuelve una clase distinta de problema y entrega información más estructurada a la siguiente. Esta separación también permite localizar errores y probar cada componente por separado.

3. El lenguaje que construiremos 01:32–02:32 | Voz: Hugo

Visual: Presentación HTML, diapositiva “MiniLang hace poco, pero lo hace de verdad”. Mostrar ejemplos válidos e inválidos.
Narración: MiniLang es deliberadamente pequeño. Solo maneja enteros de treinta y dos bits, declaraciones con la palabra `let`, operaciones de suma, resta, multiplicación y división, paréntesis y una instrucción `print`. Esa limitación es una decisión de diseño: queremos observar el proceso completo, no ocultarlo detrás de cientos de características. La gramática separa suma y multiplicación en niveles diferentes, por lo que siete por dos se agrupa antes de sumarse con seis. No incluimos funciones del usuario, cadenas, ciclos ni memoria manual. Así mantenemos el alcance verificable dentro del video.

```
let resultado = 6 * 7;
print(resultado);
```

4. Lexer: reconocer las piezas 02:32–04:02 | Voz: Hugo

Visual: Clip `LexerScene`. Luego, acercamiento a la salida real del comando `python -m minilang examples/demo.mini -tokens`.
Narración: El lexer recorre el código carácter por carácter. Cuando encuentra letras, continúa hasta formar una palabra. Después decide si esa palabra es reservada, como `let` o `print`, o si es un identificador creado por el programador. Cuando encuentra dígitos consecutivos, forma un token numérico y conserva su valor. Los símbolos de un solo carácter, como igual, asterisco o punto y coma, tienen su propio tipo.
El resultado no es todavía un árbol ni una instrucción ejecutable. Es una secuencia: `LET`, `IDENTIFIER`, `EQUAL`, `NUMBER`, `STAR`, otro `NUMBER` y `SEMICOLON`. Cada token también guarda línea y columna. Esa posición permite producir diagnósticos útiles. Si escribimos una arroba, el lexer se detiene y reporta exactamente dónde apareció el carácter que MiniLang no reconoce. Por eso un error léxico ocurre antes de que el parser intente estudiar la estructura.

5. Parser: recuperar la estructura 04:02–05:52 | Voz: Hugo

Visual: Usar primero `GrammarScene` para explicar precedencia y continuar con `AstScene`. Comparar `6 + 7 * 2` con `(6 + 7) * 2`.
Narración: El parser recibe la lista de tokens y aplica las reglas de la gramática. Una declaración comienza con `let`, continúa con un identificador, un signo igual, una expresión y termina con punto y coma. Las expresiones se dividen por precedencia. El nivel de multiplicación consume estrellas y divisiones antes de regresar al nivel de suma. Así, en seis más siete por dos, el nodo de multiplicación queda debajo del nodo de suma.
El resultado es un árbol sintáctico abstracto, o AST. Es abstracto porque conserva la información que necesitaremos más adelante y descarta detalles que ya cumplieron su propósito, como algunos paréntesis o signos de puntuación. Nuestra declaración se convierte en un nodo `LetStatement`. Ese nodo guarda el nombre `resultado` y un inicializador `Multiply` con los literales seis y siete. Si falta el punto y coma, la secuencia no coincide con la regla y obtenemos un error sintáctico. El lexer sí reconoció las piezas; el parser no logró organizarlas.

6. Semántica: comprobar el significado 05:52–07:22 | Voz: Ernesto

Visual: Clip `SemanticScene`. Mostrar tabla de símbolos y el archivo `examples/error_semantico.mini`.
Narración: Un programa puede tener una estructura válida y aun así carecer de sentido. Esta declaración respeta la gramática, pero utiliza `desconocida` antes de declararla. El análisis semántico recorre el AST y mantiene una tabla de símbolos. Cada vez que encuentra una declaración, registra el nombre, su tipo y el local de WebAssembly que le corresponde. Cuando encuentra un identificador dentro de una expresión, comprueba que ya exista.
MiniLang también rechaza una variable declarada dos veces en el mismo ámbito. Como solo manejamos enteros, todos los símbolos tienen tipo `i32`. En un lenguaje real, esta etapa incluiría reglas de compatibilidad de tipos, ámbitos anidados y firmas de funciones. La distinción importante es esta: la sintaxis pregunta si el programa tiene una forma permitida; la semántica pregunta si esa forma representa una operación válida dentro del lenguaje.

7. Del AST a WebAssembly 07:22–09:34 | Voz: Ernesto

Visual: Comenzar con AST. Hacer transición al clip `StackScene`. Mantener visibles las instrucciones actuales y la pila.
Narración: Cuando el programa supera las comprobaciones, el generador recorre el árbol y emite WebAssembly Text Format, o WAT. WebAssembly usa una máquina de pila. La instrucción `i32.const 6` coloca un seis en la pila. La siguiente coloca un siete. `i32.mul` consume ambos operandos y coloca cuarenta y dos. `local.set` retira ese valor y lo almacena en el local asociado con `resultado`. Después, `local.get` vuelve a colocar el valor en la pila y `call` invoca la función de impresión.
El orden de recorrido del AST determina el orden de estas instrucciones: primero el operando izquierdo, luego el derecho y finalmente el operador. Este esquema funciona también para árboles más profundos. WebAssembly valida que la pila y los tipos sean consistentes. Si una suma espera dos enteros y la pila no los contiene, el módulo no es válido. Nuestra implementación genera WAT, una representación textual legible. Wasmtime se encarga de analizarla, convertirla al formato interno correspondiente y ejecutar el módulo. No afirmamos haber implementado manualmente la codificación binaria de WebAssembly.

8. El contrato con el entorno 09:34–10:24 | Voz: Pablo

Visual: Clip `RuntimeScene`: validar, instanciar y ejecutar. Mostrar el módulo WASM conectado con el entorno mediante `env.print_i32`.
Narración: Hay un detalle importante: WebAssembly no incluye una instrucción universal para imprimir en una terminal. El módulo importa una función llamada `env.print_i32`. El entorno anfitrión, en nuestra demostración Wasmtime, proporciona esa función y decide qué hacer con el valor. En un navegador podría escribirlo en la página; en Python lo añadimos a una lista para verificar el resultado. Esta frontera es parte del diseño de WebAssembly: el módulo no obtiene acceso ambiental por accidente. Solo puede usar las capacidades que el anfitrión le entrega explícitamente.

9. Una optimización pequeña 10:24–11:14 | Voz: Ernesto

Visual: Clip `OptimizationScene`. Comparar WAT sin optimizar y optimizado.
Narración: Antes de generar código podemos realizar una optimización sencilla llamada *constant folding*. Como seis y siete son constantes, el compilador puede evaluar la multiplicación durante la compilación. El árbol `Multiply` se reemplaza por un literal cuarenta y dos. La salida pasa de tres instrucciones a una sola: `i32.const 42`. La regla fundamental es preservar el comportamiento observable. Por eso nuestra optimización solo evalúa operaciones constantes seguras y evita plegar una división entre cero.

10. Demostración completa 11:14–13:12 | Voz: Pablo

Visual: Usar `presentacion/index.html#12`. Ejecutar cuatro pasos: tokens, AST, WAT y salida. Después mostrar terminal con las pruebas.
Narración: Veamos el recorrido completo usando los artefactos del proyecto. Este es el programa original. Al compilar, obtenemos primero los tokens con sus lexemas. En la pestaña AST observamos la declaración, la multiplicación y la instrucción de impresión. En WAT aparecen el local, las constantes, la multiplicación y la llamada importada. Finalmente ejecutamos el módulo y obtenemos cuarenta y dos.
Ahora modificamos la expresión a seis más siete por dos. El resultado es veinte porque la multiplicación tiene mayor precedencia. Si utilizamos un nombre no declarado, el compilador no genera WebAssembly: se detiene en análisis semántico. Además de esta demostración, el repositorio contiene siete pruebas automatizadas. Cubren la ejecución correcta, la precedencia, el plegado de constantes, variables no declaradas,

declaraciones duplicadas, punto y coma faltante y caracteres desconocidos. Las pruebas conectan la explicación con comportamiento reproducible.

11. Qué construimos realmente 13:12–14:07 | Voz: Hugo

Visual: Diapositiva de alcance: Construimos / Delegamos / No incluye.
Narración: Nuestro compilador implementa las etapas suficientes para mostrar el proceso completo: análisis léxico, parsing, AST, comprobación semántica, una optimización local y generación de WAT. Delegamos el ensamblado y la ejecución a Wasmtime, de la misma manera que un compilador real utiliza herramientas y runtimes especializados. MiniLang no pretende competir con un lenguaje de producción. Su valor está en que cada transformación puede inspeccionarse, probarse y explicarse. Si quisiéramos ampliarlo, los siguientes pasos naturales serían condiciones, ciclos, funciones y un sistema de tipos más expresivo.

12. Cierre 14:07–14:36 | Voz: Todos

Visual: Montaje rápido de Código, Tokens, AST, WAT y 42. Una frase por integrante.
Pablo: Comenzamos con caracteres que por sí solos no expresaban una estructura para la máquina.
Hugo: El lexer, el parser y la semántica hicieron explícitas las piezas, sus relaciones y sus reglas.
Ernesto: El generador preservó ese significado y lo convirtió en instrucciones WebAssembly ejecutables.
Todos o voz final: Un compilador no es una caja negra: es una cadena verificable de transformaciones.

13. Créditos 14:36–14:50 | Voz: Sin narración

Visual: Usar `CreditsScene`. Su duración es 12.4 segundos y debe conservarse completa.
Audio: Música en cierre suave. Sin voz para facilitar la lectura.
Debe aparecer: Pablo Daniel Barillas Moreno; Hugo Daniel Barillas Ajín; Ernesto Ascencio; Universidad del Valle de Guatemala; Construcción de Compiladores, Sección 10; licencia Creative Commons Attribution; vínculo a fuentes y repositorio si la política del curso lo permite.

3. Mapa de animaciones Manim v02

El montaje maestro `minilang-animaciones-v02.mp4` dura 3:13.46. Los clips se insertan en los bloques indicados; no es necesario usarlos consecutivamente.

Escena	Duración	Bloque	Función narrativa
`HookScene`	16.50 s	1	Plantear la pregunta y revelar 42.
`PipelineScene`	24.90 s	2	Recorrer los contratos entre etapas.
`LexerScene`	22.23 s	4	Escanear lexemas y producir tokens.
`GrammarScene`	17.47 s	5	Explicar precedencia antes del árbol.
`AstScene`	22.00 s	5	Construir y evaluar el subárbol.
`SemanticScene`	14.47 s	6	Registrar símbolos y localizar un error.
`OptimizationScene`	15.90 s	9	Plegar constantes y comparar costo.
`StackScene`	27.00 s	7	Ejecutar seis instrucciones paso a paso.
`RuntimeScene`	20.60 s	8	Cruzar la frontera WASM–host.
`CreditsScene`	12.40 s	13	Dar crédito y permitir lectura completa.

4. Plan de grabación

Responsable	Bloques	Indicaciones
Pablo	1, 2, 8, 10 y cierre	Grabar introducción y cierre a cámara. La demostración debe capturarse en una toma de respaldo aunque después se edite.
Hugo	3, 4, 5, 11 y cierre	Mantener ritmo didáctico; hacer una pausa después de cada definición importante.
Ernesto	6, 7, 9 y cierre	Sincronizar la narración con tabla de símbolos, pila y optimización.
Equipo	Créditos y revisión	Escuchar con audífonos y altavoces; validar nombres, fechas, licencias y fuentes.

5. Lista de control editorial

- No acelerar la voz para alcanzar la duración; ajustar silencios y visuales.
- Mostrar cada fragmento de código con tamaño legible y resaltar solo la línea narrada.
- Añadir subtítulos revisados manualmente.
- Mantener música al menos 18 dB por debajo de la voz como punto de partida.
- Conservar los créditos durante diez segundos o más.
- Verificar que cualquier recurso externo permita publicación bajo CC BY.
- Publicar el video antes del 4 de noviembre de 2026 y comprobar el enlace desde otra sesión.

6. Referencias base

Referencias

[1] WebAssembly Community Group. *WebAssembly Core Specification, Release 3.0*. 2026. https://webassembly.github.io/spec/core/

[2] Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021. https://craftinginterpreters.com/

[3] Keith D. Cooper y Linda Torczon. *Engineering a Compiler*. Tercera edición, Morgan Kaufmann, 2022.

[4] Alfred V. Aho, Monica S. Lam, Ravi Sethi y Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Segunda edición, Pearson, 2006.

[5] Bytecode Alliance. *Wasmtime Documentation*. https://docs.wasmtime.dev/

[6] Manim Community. *Manim Community Documentation*. https://docs.manim.community/

[7] YouTube Help. *License types on YouTube*. https://support.google.com/youtube/answer/2797468